

# Interview Preparation Guide — Backend Developer

## 1. API Design

REST principles (resource naming, status codes, idempotency), and when GraphQL/gRPC might be preferred over REST.

## 2. Databases

Indexing strategy and its write-performance trade-offs, reading an EXPLAIN plan, normalization vs. denormalization, transactions/ACID, and NoSQL consistency models.

## 3. Authentication and Security

Session-based vs. token-based (JWT) auth, refresh tokens, OAuth 2.0 flows, and common vulnerabilities (SQL injection, CSRF, XSS) with mitigations.

## 4. System Design Basics

Caching strategies (cache-aside, write-through), load balancing basics, and handling a sudden read-traffic spike on an endpoint.

## Scenario-Based Questions

Scenario-based questions test applied judgment rather than textbook recall — interviewers are usually more interested in your reasoning process than the exact final answer.

Scenario: An API endpoint that used to respond in 100ms now takes 3 seconds under the same load. How would you investigate?

Scenario: Two concurrent requests both try to decrement the same inventory count and you're seeing occasional overselling. How do you fix this at the database/application level?

Scenario: Your service needs to call three downstream APIs to build one response, and one of them is frequently slow. How do you keep your own endpoint responsive?

## Questions by Experience Level

The bar for the same topic area rises sharply by level. Below is a representative question for a Backend Developer at each tier, showing how depth of expected answer changes — not just difficulty of the question itself.

**Intern:** Q: What does a status code like 404 or 500 actually mean?

**Entry-Level:** Q: Write an endpoint that fetches a user by ID and returns a 404 if not found.

**Associate Engineer:** Q: When would you add a database index, and when could an index hurt you?

**Engineer:** Q: Design a rate limiter for a public API used by multiple client types.

**Senior:** Q: How would you migrate a monolith's user-authentication logic into a separate service with zero downtime?

**Principal:** Q: How would you decide between a single shared database and per-service databases for a growing set of microservices, and what would change your mind?